~~Advanced~~ Number Theory 1

- Gaurish Baliga

Goal

- Factorization
- Sieve of Eratosthenes
- Prime Factorization
  - Smallest Prime Factor
  - Number of Factors
  - Sum of Factors
- Revision
  - Modular Arithmetic
  - Exponentiation

← SPF Trick

* Factorisation

n → all its factors

18 → 1 2 3 6 9 18

```
for (i = 1 ; i <= n ; i++) {
    if (n % i = 0) {
        print (i) ;
    }
}
```

$n \leq 1e5$

$\leq$

$n \leq 10^{12}$

TLE

Time complexity : $O(n)$

* Factors occur in pairs

$p \times q = n$

p: factor

q: other factor

$\rightarrow n \mid p == 0 \rightarrow p \times \frac{n}{p} = n$

* $\min(p, q) \leq \sqrt{n}$

$$p \times q = n$$

Proof: ?!

Contradiction

$\min(p, q) > \sqrt{n}$ ✗ false claime

$p \times q$

$(\sqrt{n}+1) \times (\sqrt{n}+1) = n + 2\sqrt{n} + 2$

$> n$

* Iterate over the smaller number

18

1	× 18
2	× 9
3	× 6

$\leq \sqrt{18}$

findable

$p \times q = n$

$q = \frac{n}{p}$

$i * i \leq n$

$i \leq \sqrt{n}$

```
for (i = 1 ; i*i <= n ; i++)
    n % i == 0
```

i → n/i

Factorization : Find all factors of a number N

- **Naive way**
  - Check every number for factor from 1 to N
  - Time Complexity: O(N)

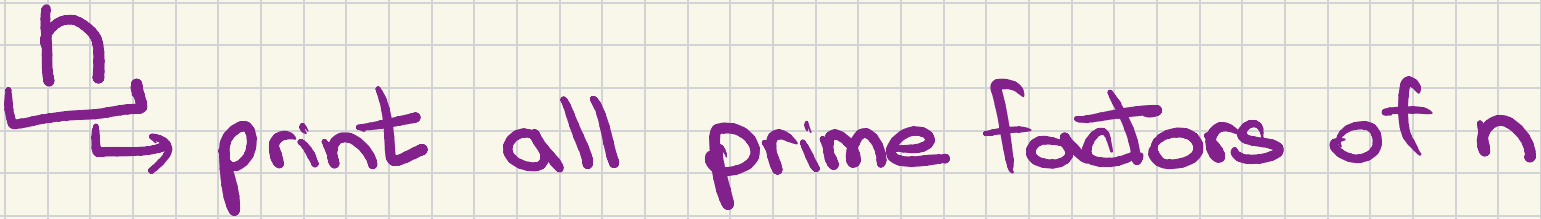
- **Factors occur in pairs**
  - N = P * Q
  - Time Complexity: Without loss of generality if P < Q => P <= sqrt(N)

- **Efficient way**
  - Check for every P from 1 to sqrt(N). Q = N / P
  - Time Complexity: O(sqrtN)

Factorization : Find all factors of a number N

Implementation (Efficient way)

```
vector<long long> findFactors(long long n) {
    vector<long long> factors;
    for (long long d = 1; d * d <= n; d++) {
        if (n % d == 0) {
            factors.push_back(d);
            if (n / d != d) // d should be different from n / d
                factors.push_back(n / d);
        }
    }
    return factors;
}
```



n = 18 → 2, 3

$O(n^2)$ → Student's idea

$O(n \times sqrt(n))$ → Mohit

```
for(i = 1; i <= n; i++) {
    if (n%i == 0 && isprime(i)){
        factors.pb(i);
    }
}
```

2 factors 1 & i itself

$O(n \times \text{sqrt}(n))$

$n \leq 10^8$

$n \leq 10^{12}$

TLE

$n = 24$

$= 2 \times 2 \times 2 \times 3$

$= 2^3 \times 3^1 = n$ prime representation

$n \neq m \Rightarrow$

primeRep(n) $\neq$ primeRep(m)

* Smallest factor of a number > 1 is always prime

Proof: Consider smallest factor is composite $x \Rightarrow$ $x \Rightarrow p_1^{k_1} p_2^{k_2} \ldots$

claim is false, hence proved

[if x is a factor of n & y is a factor of x] $\rightarrow$ y is a factor of n

$n = c_1 \times p_1 \times p_2$ ⇐ This claim does not hold

c_1: composite

↳ broken down into more primes

$$c_1 = p_1'^{k_1} p_2'^{k_2}$$

$$n = p_1'^{k_1} \times p_2'^{k_2} \times p_1 \times p_2$$

* First factor is a prime number

[* There can be only 1 prime factor $> \sqrt{n}$]

$34 = \underbrace{17^{1}}_{> \sqrt{34}} \times 2^{1}$

```
for( i=2; i*i <= n; i++ ) {
    if (n % i == 0) {
        # i is prime
        ans.pb(i);
        → while(n % i == 0) n /= i;
    }
}
```

n = 20

$2^2 \times 5^1$

2, 4, 5, 10

$$n = 2^2 \times 5^2 \times (1e^9 + 7)$$

divide by 2

new n $n/2 = 2^1 \times 5^2 \times (10^9 + 7)$

divide by 2

new $n = \underline{5^2 \times (10^9 + 7)}$

↳ $\underline{(10^9 + 7)}$ ↑ 1 or $> \sqrt{n}$

2, 5, ////////

$p > \sqrt{n}$

newn

$= 1$ ×

$> \sqrt{n}$

p^1] largest prime in n

$\not\exists\, p^k$ ✗ not possible

$p > \sqrt{n}$

$p^2 > n$ ✗ not possible

Prime Factorization

(Represent N as $p_1^{k1} p_2^{k2} .. p_m^{km}$)

- **Naive way**
  - Find all factors of N. Check each number for prime.
  - Find how many times each prime divides N.
- **Efficient way**
  - Only one of the prime factors of N can be > sqrt(N)
  - Iterate from 2 to sqrt(N)
  - Check if current number divides N. If yes, keep dividing N by that number as many times as possible.
  - Invariant: any new number that will divide N now will be a prime number.
  - Time Complexity: O(sqrtN)

Prime Factorization

Implementation (Efficient way)

distinct

```
vector<long long> primeFactorization(long long n) {
    vector<long long> factorization;
    for (long long d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            factorization.push_back(d);
            n /= d;
        }
    }
    if (n > 1) // checking for the only prime factor that is > sqrt(N)
        factorization.push_back(n);
    return factorization;
}
```

not distinct primes

Distinct

```
if (n%d == 0) {
    pb(d)   count = 0
    while(n%d==0) n/=d
```

d, count

18 = 2, 3, 3

Time complexity:

$O(\sqrt{n} + \log_2(n))$

For loop → $O(\sqrt{n})$

While loop → $O(\log_2(n))$

$\Downarrow$

$O(\sqrt{n})$

$18 \rightarrow 2 \times 3 \times 3 \rightarrow 2^1 \times 3^2$

sum of powers

$100 \rightarrow 5 \times 2 \times 5 \times 2 \rightarrow 5^2 \cdot 2^2$

Proof:

Σ powers is maximum when the prime is 2

$$n = 2^k$$

Take $\log_2$ on both sides

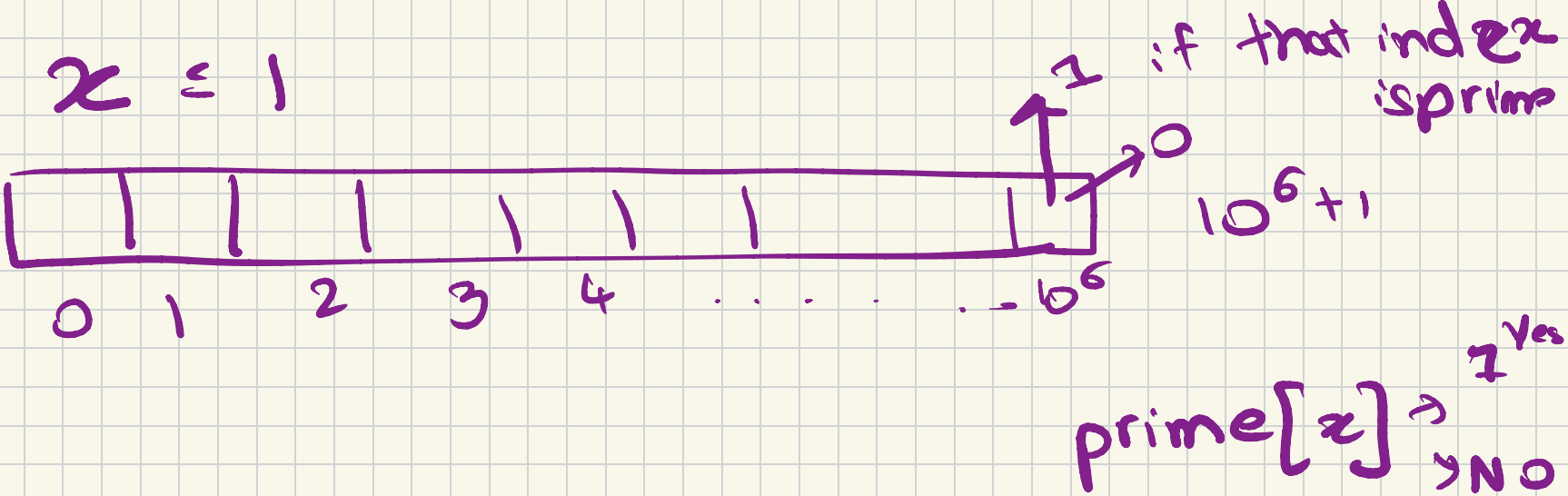
$$\log_2(n) = k$$

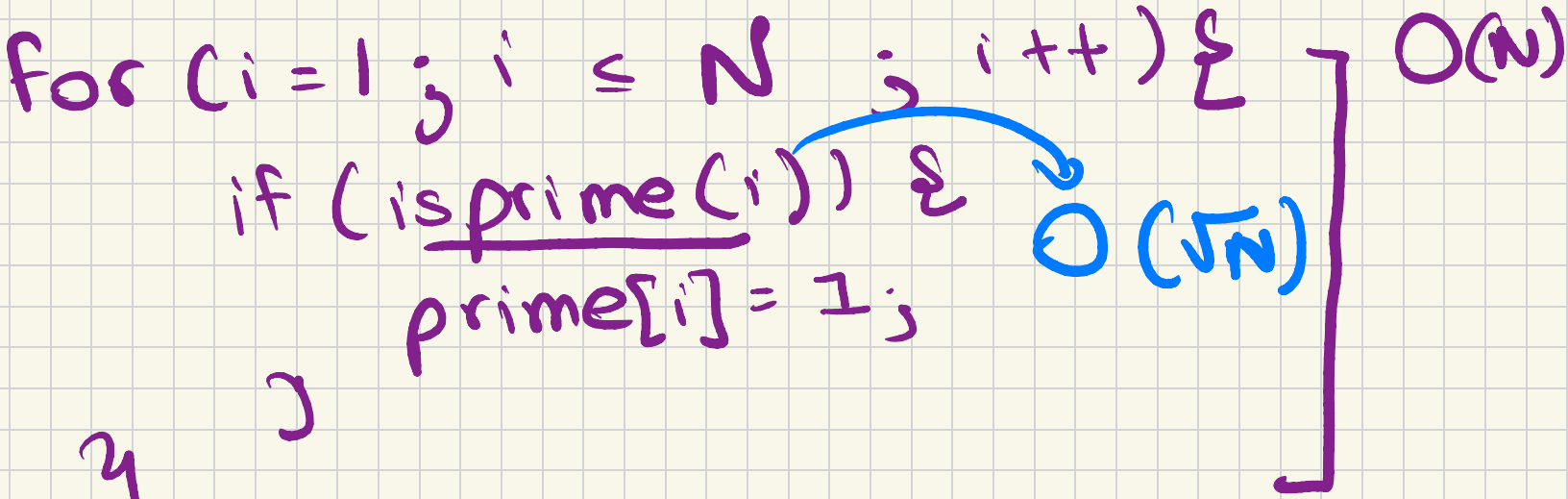
1e6 queries

→ $x \leq 10^6$ ↗ Yes → prime

↘ no → not aprime

Think!!



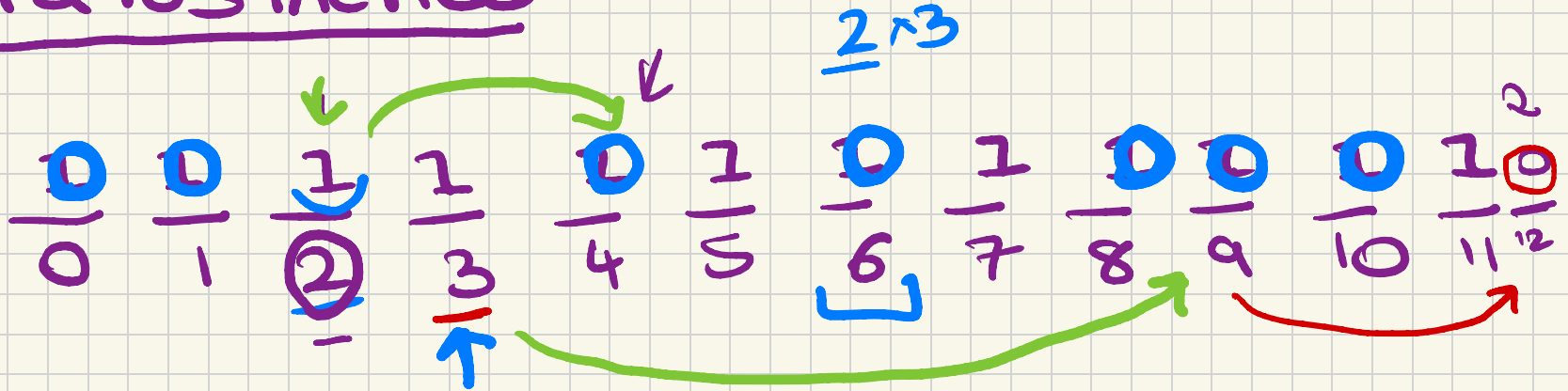


$O(N \cdot \sqrt{N}) \rightarrow 1e6 * 1e3 = 1e9$

$N = 1e6$

TLE

Eratosthenes



TC ??

```
for (i=2 ; i ≤ N ; i++) {
    if (prime[i] == 1) {
        for(j = 2*i ; j<=N ; j += i) prime[j] = 0
                ↓ i*i
    }
}
```

Proof 3

$x = p \times \underbrace{\text{something}}_{<p}$

prime p'

$p > \sqrt{x}$

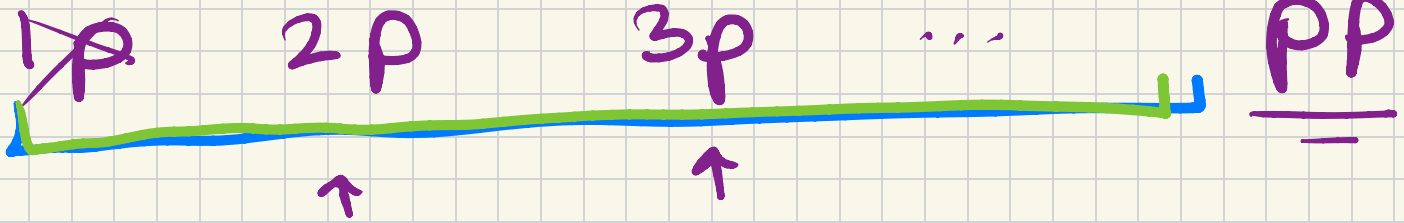
$p \to p * p$

$\to p' \leq \sqrt{x}$

consider any multiple of $p < p^2$

$p > \sqrt{x}$

$p > \sqrt{\text{multiple}}$



$$\sum \frac{N}{2} + \frac{N}{3} + \cdots\cdots\cdots \frac{N}{N}$$

↳ Harmonic Series

$O(N \log N)$ « OK ⟶ $O(N \log(\log N))$

Sieve of Eratosthenes

(Find all prime numbers from 1 to N)

- **Naive way**
  - Iterate from 1 to N
  - Check if current number is prime: O(sqrtN) or O(logN) using Miller Rabin
  - Time Complexity: O(NrootN), Space Complexity: O(1)

- **Efficient way**
  - Iterate from 2 to N
  - Keep marking numbers as non primes by iterating on multiples of primes
  - If current number (X) is unmarked -> X is Prime
  - Mark all multiples of X as non-prime
  - Time: O(Nlog(logN)), Space: O(N). Proof

Sieve of Eratosthenes

Implementation (Efficient way)

```
vector<bool> isPrime(n + 1, true);
isPrime[0] = isPrime[1] = false;

for (long long i = 2; i <= n; i++) {
    if (isPrime[i]) {
        for (long long j = i * i; j <= n; j += i) {
            // why iterate from i * i and not 2 * i
            isPrime[j] = false;
        }
    }
}
```

$i * i \leq n$

student suggested

q queries $q \leq 10^6$

↳ $x \leq 10^6$

↳ $P_1^{K_1} P_2^{K_2} \ldots$

$18 = 2 \times 3^2$

$((2,1), (3,2))$

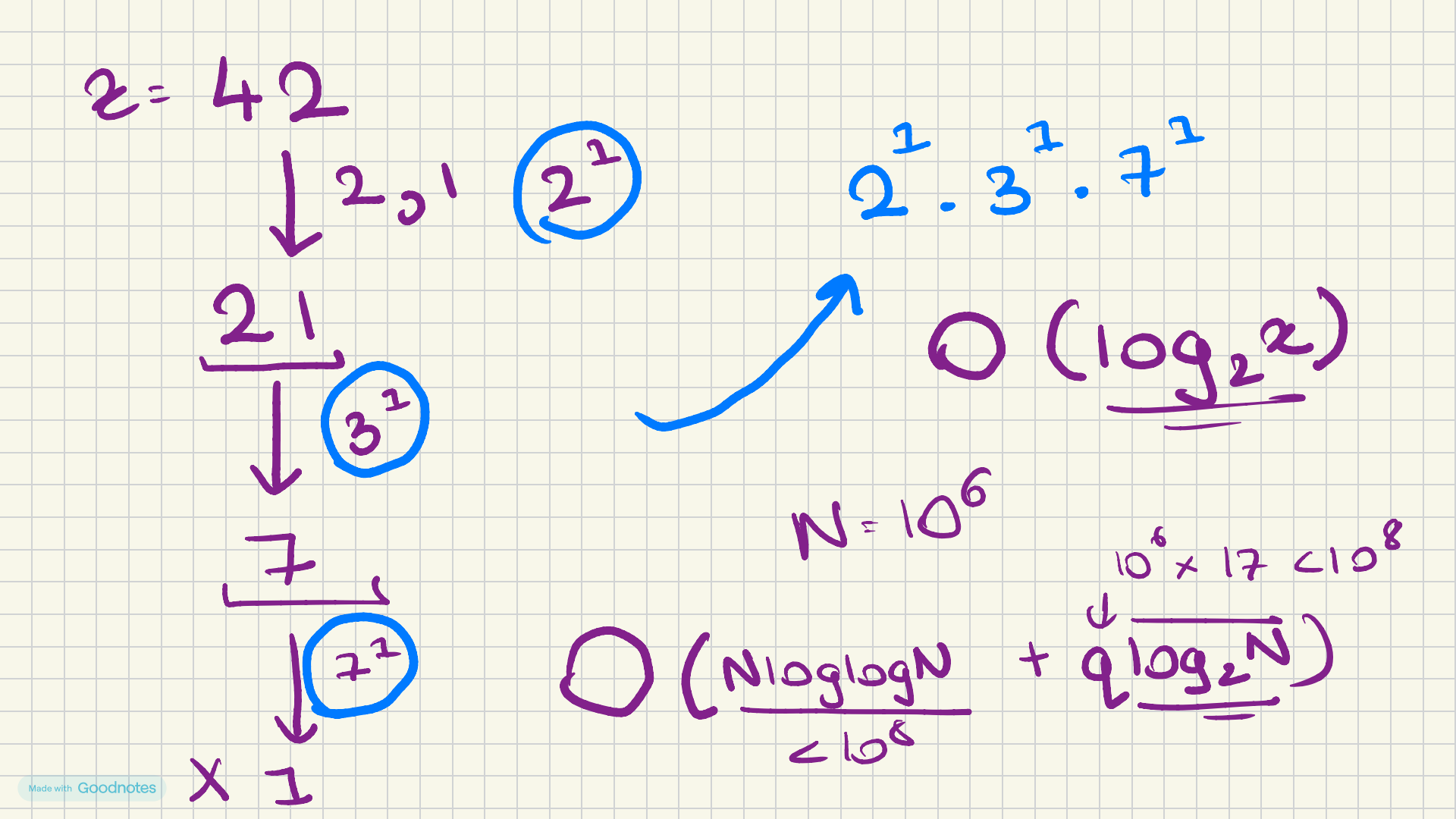
$x \rightarrow$ prime factorization

$O(q\sqrt{N}) \rightarrow 10^6 \times 10^3 = 10^9$

TLE

for all $i \leq 10^6$ we have the spf of all i

$\underline{\underline{\text{power}}} = O(\log_2 n)$


2 = 42
2, 1
2^1
21
3^1
7
7^1
X 1
$2^1 \cdot 3^1 \cdot 7^1$
$O(\log_2 2)$
$N = 10^6$
$10^6 \times 17 < 10^8$
$O(N \log\log N + q \log_2 N)$
$< 10^6$

Precomputing SPF for every N using Sieve

- **Idea**
  - Iterate through all the primes and for every multiple of that prime P, see if P is its smallest prime factor or not.
  - We can use the same idea as that in sieve to find all primes and iterate over only relevant multiples of P.
  - Time for precomputation: O(Nlog(logN))

- **Use case**
  - Finding the prime factorization of a number in O(logN) time

Efficient Prime Factorization using SPF

```cpp
vector<pair<int, int>> primeFactorization(int x, vector<int>& spf){
    vector<pair<int, int>> ans;
    while(x != 1){
        int prime = spf[x];
        int cnt = 0;
        while(x % prime == 0){
            cnt++;
            x = x / prime;
        }
        ans.push_back({prime, cnt});
    }
    return ans;
}
void solve(){
    int maxN = 1e6;
    vector<bool> isPrime(maxN, true);
    vector<int> spf(1e6, 1e9);
    for(long long i = 2; i < maxN; i++){
        if(isPrime[i]){
            spf[i] = i;
            for(long long j = i * i; j < maxN; j += i){
                isPrime[j] = false;
                spf[j] = min(spf[j], (int)i);
            }
        }
    }
    vector<pair<int, int>> primeF = primeFactorization(36, spf);
}
```

spf = smallest prime factor

1. n → prime factorization representation

$$n = p_1^{k_1} \; p_2^{k} \; p_3^{k_3} \; \dots \; m$$

p	power
2	3
(1e9+7)	10^6

$p_1 \quad k_2$

$p_2 \quad k_2$

$p_m \quad k_m$

1. number of divisors
2. sum of divisors

2 1
3 2

1, 2, 3, 6, 9, 18 → 6
1+2+3+6+9+18 → 42

$$n = p_1^{k_1} p_2^{k_2} p_3^{k_3} \cdots$$

$$18 = 2^1 \cdot 3^2$$

3^0

3^1

3^2

2^0 2^1

$(1+1) * (2+1) = 2 * 3 = 6$

$1 = 2^0 \cdot 3^0$

$2 = 2^1 \cdot 3^0$

$3 = 2^0 \cdot 3^1$

$9 = 2^0 \cdot 3^2$

$6 = 2^1 \cdot 3^1$

$18 = 2^1 \cdot 3^2$

Sum of divisors

#1 Hard

18

$$= 1 + 2 + 3 + 6 + 9 + 18$$

$$= 2^0 3^0 + 2^1 3^0 + 2^0 3^1 + 2^1 3^1 + 2^0 3^2 + 2^1 3^2$$

$$= 2^0 (3^0 + 3^1 + 3^2) + 2^1 (3^0 + 3^1 + 3^2)$$

$$= \underbrace{(2^0 + 2^1)} \times \underbrace{(3^0 + 3^1 + 3^2)}$$

$2^0 + 2^1 + 2^2 \cdots 2^x$ — GP

↑ GP

Number and Sum of Divisors:

https://www.spoj.com/problems/DIVSUM/

HW
↳ product of divisors

- $N = p_1^{e1} p_2^{e2} \ldots\ldots p_k^{ek}$
  - **Number of Divisors:**

$$d(n) = (e_1 + 1) \cdot (e_2 + 1) \cdots (e_k + 1)$$

  - **Sum of Divisors:**

$$\sigma(n) = \left[\frac{p_1^{e_1+1} - 1}{p_1 - 1}\right] \left[\frac{p_2^{e_2+1} - 1}{p_2 - 1}\right] \cdot \left[\frac{p_k^{e_k+1} - 1}{p_k - 1}\right]$$

Modular Arithmetic

- $(A + B) \% M = [(A \% M) + (B \% M)] \% M$
- $(A - B) \% M = [(A \% M) - (B \% M) + M] \% M$
- $(A * B) \% M = [(A \% M) * (B \% M)] \% M$
- $(A / B) \% M = [(A \% M) * (B^{-1} \% M)] \% M$

What is $B^{-1} \% M$? Modular Inverse (Will be covered later)

Binary Modular Exponentiation

Idea:

$$a^n = \begin{cases} 1 & \text{if } n == 0 \\ \left(a^{\frac{n}{2}}\right)^2 & \text{if } n > 0 \text{ and } n \text{ even} \\ \left(a^{\frac{n-1}{2}}\right)^2 \cdot a & \text{if } n > 0 \text{ and } n \text{ odd} \end{cases}$$

Time Complexity: O(log(n))

Binary Modular Exponentiation

Implementation: (Don’t forget to take MOD when mentioned)

Recursive

```
long long binpow(long long a, long long b) {
    if (b == 0)
        return 1;
    long long res = binpow(a, b / 2);
    if (b % 2)
        return res * res * a;
    else
        return res * res;
}
```

Iterative

```
long long binpow(long long a, long long b) {
    long long res = 1;
    while (b > 0) {
        if (b & 1)
            res = res * a;
        a = a * a;
        b >>= 1;
    }
    return res;
}
```

Important Links

These links

- **Factorization:** https://cp-algorithms.com/algebra/factorization.html
- **Binary Exponentiation:** https://cp-algorithms.com/algebra/binary-exp.html#algorithm
- **Number Theory:** https://www.hackerearth.com/practice/math/number-theory/basic-number-theory-1/tutorial/
- **Sieve:** https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html